

shared e skills-ddd-clean sob Análise

Viável trazer pra este projeto? Criar submodule próprio? Juntar no token-economy?

Agosto de 2026

Sumário

1. O que são, de verdade	1
1.1 shared	1
1.2 skills-ddd-clean	1
2. Viável trazer para este projeto (proj_fabrica-comunicacao)?	1
3. Criar um submodule seu com essas skills — viável?	2
4. Juntar essas skills dentro do token-economy-shared?	2
5. Opinião final, sem filtro	3
6. Fontes	3

1. O que são, de verdade

Ambos são **forks** de repositórios da conta *mentoria-360* (não são originais seus) — provavelmente material de um programa de mentoria que você acompanha.

1.1 shared

Biblioteca **TypeScript** de blocos de construção DDD: Result, Entity, AggregateRoot, ValueObject, UseCase, eventos de domínio, contratos de repositório, sistema de erros estruturados. Publicada como pacote npm (@namespace/shared), compilada com tsc, testada com Jest. É **código executável de runtime**, não documentação — para usar, você importa as classes num projeto Node/TypeScript.

1.2 skills-ddd-clean

Coleção de **skills de agente de IA** para padronizar arquitetura em projetos com essa stack específica: bootstrap de monorepo **TurboRepo**, modelagem de domínio, camada de aplicação, persistência com **Prisma**, leitura CQRS, front-end em **React** (sufixos de arquivo .component.tsx, .entity.ts, .controller.ts, .module.ts, .model.prisma — tudo nomenclatura Nest/React/Prisma). O próprio README diz: “desenhado para ser reutilizado como Git submodule em outros repositórios” — ou seja, o propósito de ser submodule já é intencional no design deles, só que para **esse** tipo de projeto.

2. Viável trazer para este projeto (proj_fabrica-comunicacao)?

Não. Incompatibilidade de stack total, não parcial.

- Este projeto é **Python** (FastAPI, scripts, pytest) + **HTML/CSS/JS estático** (painel) + **pandoc/typst** (geração de PDF). Não tem NestJS, não tem Prisma, não tem React, não tem TurboRepo, não tem TypeScript em lugar nenhum.

- `shared` é uma **biblioteca de runtime JS/TS compilada** — não existe forma de “usar” isso em código Python sem reescrever do zero, o que anula completamente o propósito de reuso (reescrever do zero não é reuso).
- `skills-ddd-clean` é uma coleção de skills cujo valor inteiro está em convenções de nomenclatura de arquivo de uma stack (`.entity.ts`, `.controller.ts`, `.model.prisma`) que **não existe** neste repo. Aplicado aqui, cada skill instruiria o agente a criar arquivos `.ts/.tsx` num projeto que não roda JavaScript nenhum.

Isso não é “sobreposição parcial, algumas skills servem” (como no caso do Agent Skills genérico) — é **incompatibilidade de domínio técnico completa**. Diferente do LoopX ou do Agent Skills, aqui nem vale a pena extrair um subconjunto: não existe subconjunto aplicável.

3. Criar um submodule seu com essas skills — viável?

Depende do que “seu” significa aqui:

- **Tecnicamente trivial:** transformar os forks em repositórios independentes (não-fork) sob sua conta é uma operação de minutos — `git clone`, remover o histórico de fork, recriar como repo próprio (ou simplesmente continuar usando o fork como está, que já funciona igual a um submodule normal). Não há trabalho de engenharia real aqui.
- **Só faz sentido se você tiver projetos reais nessa stack.** E você tem: verifiquei sua conta e `astracampaign`, `Cadastrros-Conexao`, `flowChat` e `conexao-hub-main` são todos **TypeScript**. Nesses projetos, sim, `shared` (biblioteca) + `skills-ddd-clean` (skills de padronização) fazem sentido genuíno como submodule — é exatamente o caso de uso para o qual foram desenhados.

Minha recomendação: não crie nada de novo agora — os forks já cumprem o papel de “seu submodule” tecnicamente. Se quiser independência total do upstream `mentoria-360` (poder editar sem risco de conflito num futuro `git pull`), aí sim vale “promover” o fork a repositório próprio — mas isso é uma decisão de manutenção, não de viabilidade técnica (já é viável hoje).

4. Juntar essas skills dentro do `token-economy-shared`?

Não recomendo — mesma razão já usada na crítica ao Agent Skills. `token-economy-shared` tem identidade estreita e deliberada: economia de token para Claude Code, independente de linguagem/stack do projeto consumidor. Misturar skills de arquitetura DDD/TypeScript lá dentro:

- quebra a promessa do nome do repo para qualquer outro projeto seu que reuse `token-economy-shared` só querendo economia de token;
- forçaria projetos Python (como este) a ganhar skills TypeScript irrelevantes só por estarem no mesmo submodule;
- mistura dois eixos ortogonais (economia de token é sobre *como* o agente se comunica; DDD/Clean Architecture é sobre *que* código o agente escreve num domínio específico) — são preocupações independentes que merecem repos independentes.

Se você quiser um terceiro submodule dedicado a “padrões de arquitetura TypeScript/DDD” reunindo `shared` + `skills-ddd-clean` num único lugar coeso, isso é razoável — mas como **repo novo e próprio**, não dentro do `token-economy-shared`.

5. Opinião final, sem filtro

- **Para proj_fabrica-comunicacao: não usar, em nenhuma forma.** Stack incompatível de ponta a ponta; não há meio-termo aqui como houve com o Agent Skills genérico.
 - **Os forks já são funcionalmente “seu submodule”** — não há trabalho real a fazer para torná-los usáveis; a única decisão pendente é se vale desacoplar do upstream `mentoria-360` por controle, o que é opcional.
 - **Não junte no token-economy-shared** — identidade e propósito diferentes, misturar prejudica os dois.
 - **Onde de fato aplicar:** `astracampaign`, `Cadastros-Conexao`, `flowChat`, `conexao-hub-main` — todos TypeScript, todos candidatos reais e imediatos para adicionar `shared` e `skills-ddd-clean` como submodule, se esses projetos seguem (ou vão seguir) arquitetura DDD/Clean.
-

6. Fontes

- `Heverton-web/shared` (fork de `mentoria-360/shared`)
- `Heverton-web/skills-ddd-clean` (fork de `mentoria-360/skills-ddd-clean`)
- Listagem de repositórios da conta `Heverton-web` (via `gh repo list`), usada para identificar os projetos TypeScript candidatos na seção 3.